

```
main() {
```

≡≡≡ 3 lines

create Task → runnable

```
Thread t = new Thread( new Runnable() {
```

```
public void run() {
```

```
    print 100 lines
```

```
}
```

```
});
```

```
t.start();
```

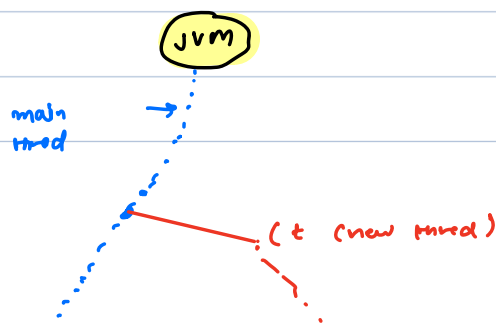
(main
 inner thread)

10 lines

3

(1) inner thread prints few lines and terminate when the main thread is exit.

(2) inner thread is completely executed even if the main thread is exit.



...
= exit
=

"in ... " (complete)

- when we exit from main thread then all the thread is also terminated.

by default threads are non daemon.

we set all the threads to daemon to achieve this.

t.setDaemon(true); // daemon

by default → t.setDaemon(false); // non daemon.

Synchronization

```
class Amount {
    int val;
```

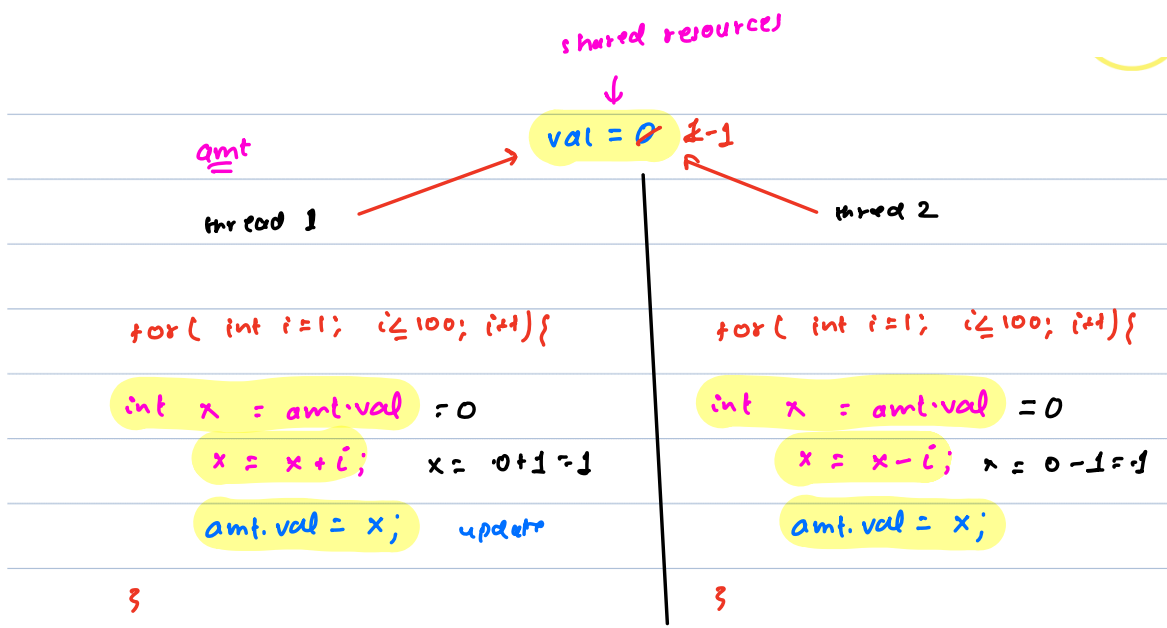
```
    thread
class incre {
    for(int i=1 → i<1000){
        val += i;
    }
}
```

```
    thread
class decre {
    for(int i=1 → i<1000){
        val -= i;
    }
}
```

expected value is

zero.

we get different value each time.

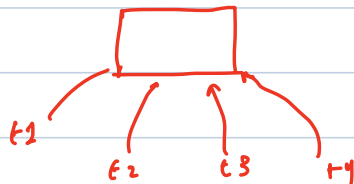


when two or more thread are trying to accessing the same shared resource then this scenario called synchronization

what are the problem we face :

1) Race condition :- it is occur when multiple threads access shared resources at same time.

it leads to inconsistent or unexpected result.



2) Critical section:

A critical section is a segment of code in program that access shared resources.

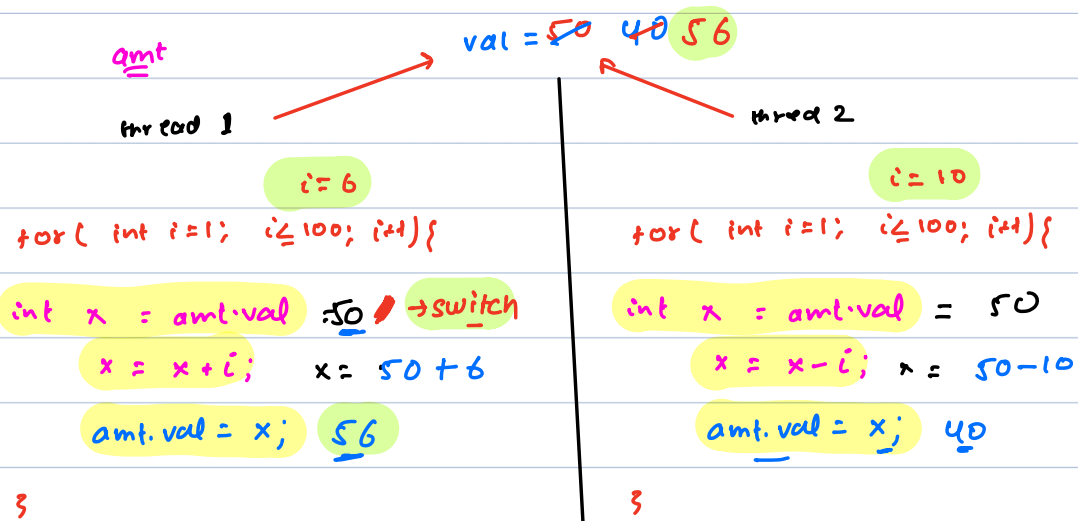
```

1      syso ('a')
2      for (int i=1; i<=100; i++){
3          int x = amt.val = 0
4          x = x + i;    x = 0+1 = 1
5          amt.val = x;  update
6      }
7
8      syso ("b");
  
```

The code block from line 3 to line 5 is enclosed in a pink box and labeled "critical section".

3) Preemptive → pause and switch to next

if the critical part of a program behave preemptive.



it should not be preemptive

solution

synchronization

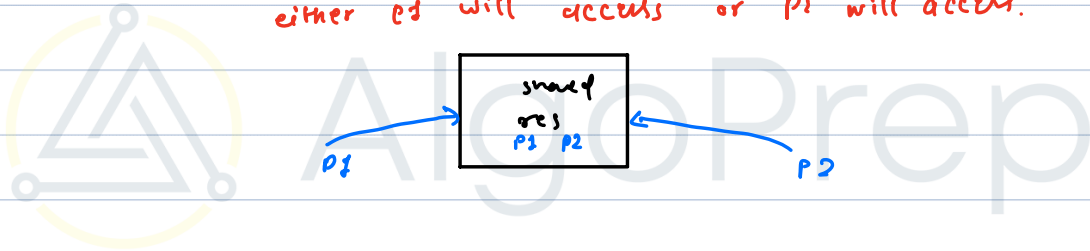
mechanism.

break till 9:20

what are the property for good mechanism

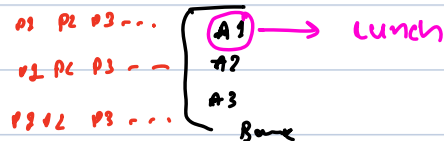
- 1) **mutual exclusion**: it ensure that only one thread can access the critical section at any given time.

either p1 will access or p2 will access.



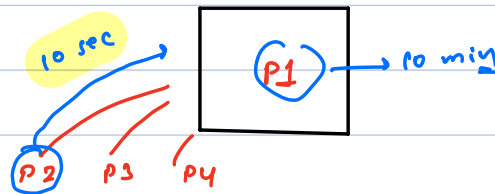
- 2) **Progress** or **liveness** it refers to a property that a system should continue to make progress.

they are waiting



- 3) **non busy waiting** thread doesn't continuously

check for a condition to become true.



P2 knock the door every 10 sec.

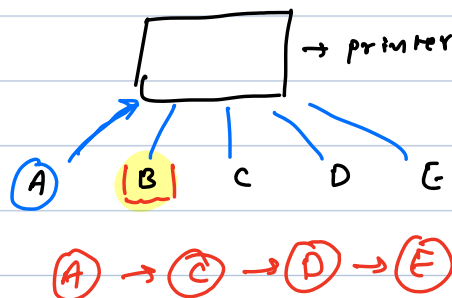
```
while (room is not empty) {  
    knock the door  
    waiting  
}
```

busy waiting

it is not an efficient way.

- thread wait for notifications or signal to proceed their execution instead of continuously checking for a condition to become true.

4) bounded wait or fairness



it ensure that a thread does not have to wait indefinitely to enter a

critical section.

- go to critical section in the order the threads came.

Synchronization mechanism

1) mutex it is implements locks

mutual execution

- A thread must take a lock before entering in critical section.

- before exit from the critical section we should unlock it.

- it acts as a gatekeeper.

i = 6

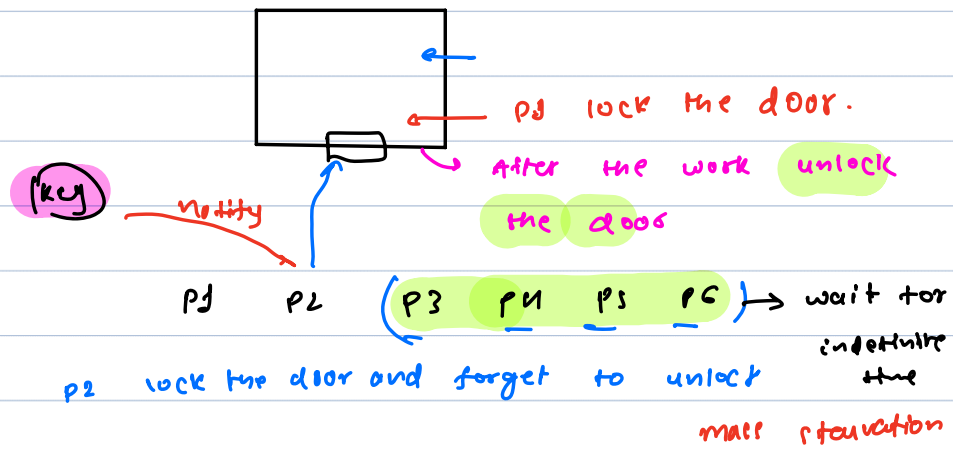
```
for (int i = 1; i ≤ 100; i++) {
```

```
    lock();
```

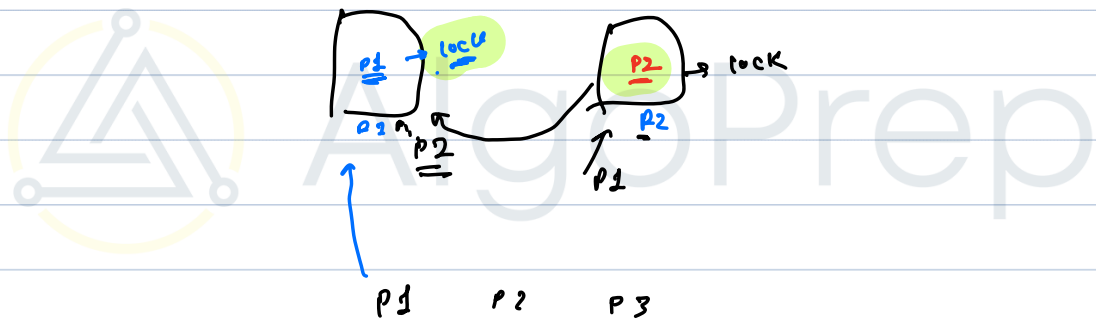
```
    {  
        int x = amt.val  
        x = x + i;  
        amt.val = x;  
    } → critical section
```

```
    unlock();
```

```
}
```



or deadlock:



when two threads are waiting to each other then it is called deadlock.

lock the critical section and before exit unlock it.

```
synchronised ( ) {
    critical section
}
```

```

4 public class task3 {
    Run | Debug
5     public static void main(String[] args) {
6         System.out.println(x:"main thread started...");
7
8         Thread t = new Thread(new Runnable(){
9             public void run(){
10                 for(int i = 1; i<=100; i++){
11                     System.out.println("I am inner Thread "+i);
12                 }
13             }
14         });
15
16
17         t.setDaemon(on:true);
18         t.start();
19         for(int i = 1; i<=10; i++){
20             System.out.println("I am main Thread "+i);
21         }
22
23     }
24 }
25

```

```

39 public class task4 {
    Run | Debug
40     public static void main(String[] args) throws InterruptedException {
41         Amount amt = new Amount();
42         Lock l1 = new ReentrantLock();
43
44
45         Inc tsk1 = new Inc(amt , l1);
46         Dec tsk2 = new Dec(amt , l1);
47
48         Thread t1 = new Thread(tsk1);
49         Thread t2 = new Thread(tsk2);
50
51         t1.start();
52         t2.start();
53
54
55         Thread.sleep(millis:1000);
56         // main thread
57         System.out.println("after performing both task "+ amt.val);
58     }
59 }
60 }
61

```

```
1  import java.util.concurrent.locks.Lock;
2  import java.util.concurrent.locks.ReentrantLock;
3
4  class Amount{
5      int val;
6  }
7
8  class Inc implements Runnable {
9      private Amount amt;
10     private Lock l;
11     public Inc(Amount a , Lock l){
12         this.amt = a;
13         this.l = l;
14     }
15     public void run(){
16         for(int i = 1; i<=10000; i++){
17             l.lock();
18             amt.val += i;
19             l.unlock();
20         }
21     }
22 }
23 class Dec implements Runnable {
24     private Amount amt;
25     private Lock l;
26     public Dec(Amount a , Lock l){
27         this.amt = a;
28         this.l = l;
29     }
30     public void run(){
31         for(int i = 1; i<=10000; i++){
32             l.lock();
33             amt.val -= i;
34             l.unlock();
35         }
36     }
37 }
38
```
